LEAF HEALTH CHECK

Team ID: SWTID-2026-9257
Team Size: 5
Team Leader: Asha S
Team Member: Nivethitha P
Team Member: Sridevi M
Team Member: Meenakshi M
Team Member: Keerthana S

Introduction:

Plant health monitoring plays a vital role in agriculture, horticulture, and home gardening. Healthy plants are essential for producing high-quality crops and maintaining sustainable agricultural practices. However, plants are often affected by various diseases caused by fungi, bacteria, viruses, and environmental conditions. These diseases usually appear first on plant leaves in the form of discoloration, spots, or texture changes.

Traditionally, plant disease detection is performed through manual inspection by farmers or agricultural experts. This process requires experience and knowledge to correctly identify the symptoms of different plant diseases. In many cases, early signs of infection such as yellowing or small brown spots are overlooked. As a result, the disease spreads and causes severe damage to crops, leading to reduced productivity and financial loss.

To address this challenge, modern technology and artificial intelligence can be used to detect plant diseases automatically. Image processing and machine learning techniques can analyze plant leaf images and identify disease patterns quickly and accurately. This approach reduces the need for expert knowledge and enables early detection of plant diseases.

The **Leaf Health Check** project is developed as an intelligent application that analyzes plant leaf images and provides instant health diagnosis. The system uses image processing, computer vision, and artificial intelligence techniques to detect discoloration patterns on leaves and predict possible

diseases. The application also determines the severity level of the disease and provides helpful recommendations to improve plant health.
The project is implemented using the **Streamlit framework**, which allows the development of an interactive web interface. Users can upload images of plant leaves, and the system processes the image using AI algorithms to generate a detailed diagnosis report.

Objective of the Project

The main objective of the Leaf Health Check system is to develop an intelligent platform that can automatically detect plant diseases from leaf images.
The specific objectives include:

- To build a user-friendly web application for plant health analysis
- To allow users to upload plant leaf images for examination
- To analyze leaf discoloration patterns using image processing techniques
- To identify plant species and possible diseases using AI models
- To classify the severity of leaf damage
- To provide rescue recommendations for improving plant health

This system helps farmers, students, and gardeners make informed decisions about plant care.

System Overview

Leaf Health Check is an AI-powered system that processes images of plant leaves and generates health analysis results. The application integrates several technologies including **computer vision, machine learning, and web-based interfaces**.
The user interacts with the system through a **Streamlit web interface**. The interface allows users to upload images of plant leaves. Once the image is uploaded, the system performs several processing steps such as resizing, normalization, and noise removal.
After preprocessing the image, the system analyzes the leaf for discoloration patterns such as yellowing or browning. These visual symptoms are indicators of possible plant diseases or nutrient deficiencies.

The processed image is then passed to an AI model that predicts the plant species and identifies possible diseases. The system also calculates a confidence score indicating how accurate the prediction is.
Based on the amount of discoloration detected, the system determines the severity level of the disease. The severity level ranges from **Healthy to Dying** depending on the percentage of affected leaf area.
Finally, the system generates a diagnosis report that includes the plant name, predicted disease, severity level, confidence score, and three rescue tips to help improve plant health.

Use Case Scenarios

Scenario 1: Leaf Image Upload and Health Analysis
Users upload a plant leaf image through the Streamlit interface. The system preprocesses the image by enhancing brightness, resizing it, and removing noise for accurate analysis. The AI model then detects discoloration patterns and predicts whether the leaf is healthy or affected by a disease. The result is displayed along with the severity level and confidence score.
Scenario 2: Plant Species Identification and Disease Detection
The system matches the uploaded leaf image with a plant species database. After identifying the plant type, it analyzes common diseases associated with that species and predicts the most probable condition. This ensures that the diagnosis is plant-specific and more accurate.
Scenario 3: Severity Grading and Rescue Recommendations
Based on the percentage of damaged area and the type of discoloration detected, the system assigns a severity level ranging from Healthy to Dying. It then generates three intelligent rescue tips that help users take immediate action to improve plant health and prevent further damage.

System Architecture:

The architecture of the Leaf Health Check system consists of several components that work together to perform plant health analysis.

The process begins with the **user uploading an image** through the Streamlit interface. The uploaded image is then passed to the **image processing module**, where it undergoes several preprocessing steps. These steps include resizing the image, converting it into RGB format, and removing noise to improve analysis accuracy.

Next, the processed image is analyzed by the **AI model layer**. The AI model is typically a convolutional neural network (CNN) trained on plant disease datasets. This model identifies the plant species and predicts possible diseases based on visual patterns found in the leaf image.

After the disease prediction step, the system calculates the **severity grading** by measuring the percentage of discolored pixels in the leaf image. The severity level helps determine how serious the disease is.

The system also connects to a **plant species database**, which stores information about plants, common diseases, and treatment methods. This database helps provide accurate recommendations based on the detected plant species.

Finally, the **recommendation engine** generates rescue tips for the user. These tips help users take corrective actions such as improving watering conditions, applying fertilizers, or adjusting sunlight exposure.

The results are then displayed to the user through the Streamlit interface in a structured diagnosis report.

```
+-------------------+
  |      User       |
  +-------------------+
          |
          v
  +-------------------+
  |   Streamlit UI    |
  |  (Image Upload)   |
  +-------------------+
          |
          v
  +-------------------+
  |  Image Processing |
  |     (OpenCV)      |
  | Resize, Filter    |
  | Discoloration     |
```

```
+-------------------+
        |
        v
+-------------------+
|     AI Model      |
| (CNN / Gemini AI) |
| Disease Detection |
+-------------------+
        |
        v
+-------------------+
|  Severity Engine  |
| Damage Percentage |
| Health Grading    |
+-------------------+
        |
        v
+-------------------+
| Recommendation    |
|     Engine        |
| Rescue Tips       |
+-------------------+
        |
        v
+-------------------+
|  Diagnosis Report |
| Plant + Disease   |
| Severity + Tips
+-------------------+
```

Technologies Used:

The Leaf Health Check project uses several technologies to build the complete system.

Python is used as the primary programming language for implementing the backend logic and AI integration. Python is widely used in data science and machine learning applications.

Streamlit is used to develop the web interface. It allows developers to build interactive web applications using simple Python scripts. Streamlit makes it easy for users to upload images and view results in real time.

OpenCV is used for image processing tasks. It helps perform operations such as image resizing, noise reduction, and color detection.

TensorFlow or PyTorch is used as the deep learning framework for training and running the plant disease classification model.
NumPy and Pandas are used for numerical calculations and data processing.
Google Gemini API may also be used for advanced image analysis and AI-based diagnosis generation.
SQLite or PostgreSQL can be used as the database for storing plant species information and disease data.

Pre-requisites

1. Python 3.8 or above
2. Streamlit Framework
3. TensorFlow / PyTorch
4. OpenCV
5. Required Libraries (requirements.txt)

Example installation:

```
pip install -r requirements.txt
```

Run the application:

```
streamlit run app.py
```

Image Processing and Disease Detection

Image processing plays an important role in detecting plant diseases. When a leaf image is uploaded, the system performs several preprocessing operations.
The image is first resized to ensure consistent input dimensions for the AI model. This step ensures that the model can process images efficiently.
Next, the image is converted into RGB format. RGB representation allows the system to analyze color patterns more effectively.

Noise reduction techniques such as filtering are applied to remove unwanted image artifacts. This improves the clarity of the image and increases the accuracy of the analysis.
The system then performs **discoloration detection** by identifying yellow and brown pixels in the leaf image. These colors are common indicators of plant diseases or nutrient deficiencies.
By calculating the number of discolored pixels relative to the total number of pixels in the leaf image, the system determines the percentage of leaf damage.

Severity Grading Logic

The severity grading system is used to classify the health condition of the plant leaf.
The system calculates the percentage of discolored regions on the leaf. Based on this percentage, the system assigns a severity level.
The grading scale is defined as follows:

- 0–10% discoloration – Healthy
- 11–30% discoloration – Mild infection
- 31–60% discoloration – Moderate infection
- 61–80% discoloration – Severe infection
- 81–100% discoloration – Dying leaf

This classification helps users understand the seriousness of the plant disease and take appropriate action.

Recommendation Engine:

After detecting the disease and severity level, the system generates rescue recommendations. These recommendations are designed to help users improve plant health and prevent further damage.
The recommendation engine uses information stored in the plant database and AI-generated insights to produce practical suggestions.
Typical rescue tips may include:

- Removing infected leaves to prevent disease spread
- Applying suitable fertilizers or fungicides
- Adjusting watering schedules

- Improving sunlight exposure
- Ensuring proper air circulation around the plant

These recommendations help users take immediate action to protect their plants.

User Interface

The user interface of the Leaf Health Check system is developed using Streamlit. The interface is designed to be simple, clean, and easy to use.
The main page of the application displays the project title and instructions for uploading leaf images. Users can upload images in formats such as JPG, PNG, or JPEG.
Once the image is uploaded, the system displays a preview of the leaf image. Users can then click the **Analyze** button to start the analysis process.
During processing, the system shows a loading indicator to inform users that the analysis is in progress.
After analysis is completed, the system displays the diagnosis results including the plant species, predicted disease, confidence score, severity level, and rescue tips.
The interface may also provide an option to download the diagnosis report as a text or PDF file.

Testing and Validation

Testing is an important stage in ensuring that the system works correctly.
The Leaf Health Check application was tested using different types of leaf images.
Test cases included healthy leaves, leaves with yellow discoloration, leaves with brown spots, and severely damaged leaves.
The system was also tested with low-quality or blurred images to evaluate how well it handles poor input data.
The results were reviewed to verify that the disease predictions, severity grading, and rescue recommendations were accurate and consistent.
Performance optimization techniques were applied to improve processing speed and reduce response time.

Example Output 1 – Healthy Leaf
Input: Image of a healthy green leaf
Diagnosis Report
Plant Name: Mango
Disease: No Disease Detected
Confidence Score: 95%
Severity Level: **Healthy**
Rescue Tips:

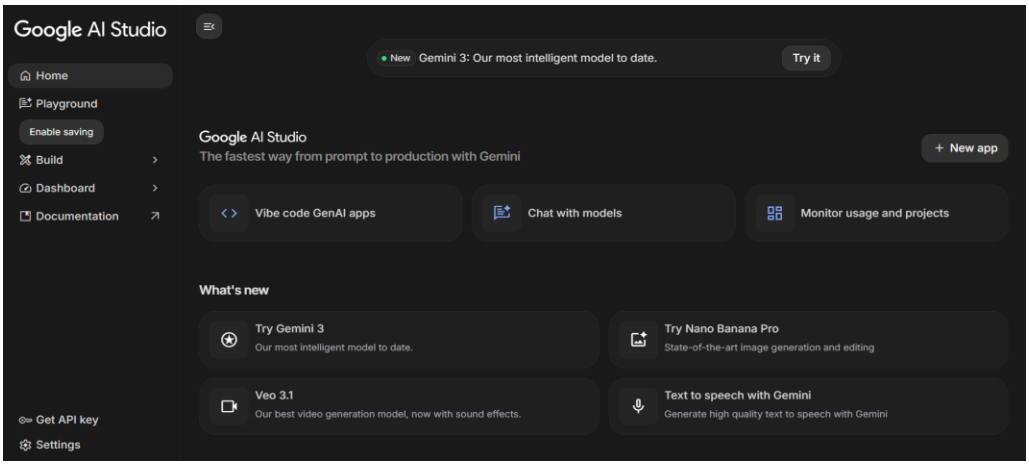
1. Maintain regular watering schedule.
2. Ensure adequate sunlight for proper growth.
3. Continue using balanced fertilizers to support plant health.

MILESTONE 1: Gemini AI Initialization

MILESTONE 1: AI Model Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required for the **Leaf Health Check** system. The platform utilizes a deep learning–based computer vision model to analyze plant leaf images, detect discoloration patterns, identify plant species, and generate health diagnosis with rescue recommendations

Setup Development Environment and AI Model



1. Install Required Software

- Install Python (version 3.8 or above)
- Install required IDE (VS Code / PyCharm)
- Setup virtual environment for project isolation

2. Install Required Libraries

```
pip install streamlit opencv-python tensorflow numpy pandas pillow python-dotenv
```

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core application using the **Streamlit framework** integrated with the **Google Gemini API**. The system accepts plant leaf images, processes visual information, analyzes discoloration patterns, and generates an AI-based diagnosis with severity grading and rescue recommendations.

```
Leaf-Health-Check/
│
├── app.py                  # Main Streamlit application
├── requirements.txt        # Project dependencies
├── .env                    # Environment variables
│
├── model/
│   └── preprocess.py       # Image preprocessing logic
│
├── utils/
│   ├── gemini_api.py       # Gemini API integration
│   ├── severity.py         # Severity grading logic
│   ├── recommendations.py  # Rescue tips generator
│
├── assets/
│   └── style.css           # UI styling
│
└── outputs/                # Generated reports
```

Utility Functions

Clean AI Response

```python
def clean_response(text):
    return text.strip()
```

Unique File Generator

```python
from datetime import datetime

def generate_filename():
    return datetime.now().strftime("%Y%m%d_%H%M%S")
```

MILESTONE 3: Backend – Streamlit Integration (app.py)

This milestone focuses on implementing the complete backend logic inside the **Streamlit application**, integrating the **Google Gemini AI Studio API**, handling image processing, performing diagnosis generation, and enabling export functionality. The system ensures proper validation, structured responses, and smooth interaction between the UI and AI services.

Activity 3.1: Complete Function Definitions

All backend processing is implemented with:

- Input image validation
- Error handling mechanisms
- Gemini API response handling
- Logging and debugging support
- Structured diagnosis output formatting

Image Validation Example

```
def validate_image(file):
    if file is None:
        return False, "No image uploaded"
    return True, "Valid image"
```

Modular Architecture

Main Application Structure

```
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")

st.title("🌿 Leaf Health Check System")

print("="*60)
print("Leaf Health Check - Starting Application")
print("✓ Gemini API Key configured")
print("✓ Using model: gemini-2.5-flash")
print("="*60)
```

The Streamlit application directly manages:

- User interaction
- AI communication
- Diagnosis generation

- Result visualization

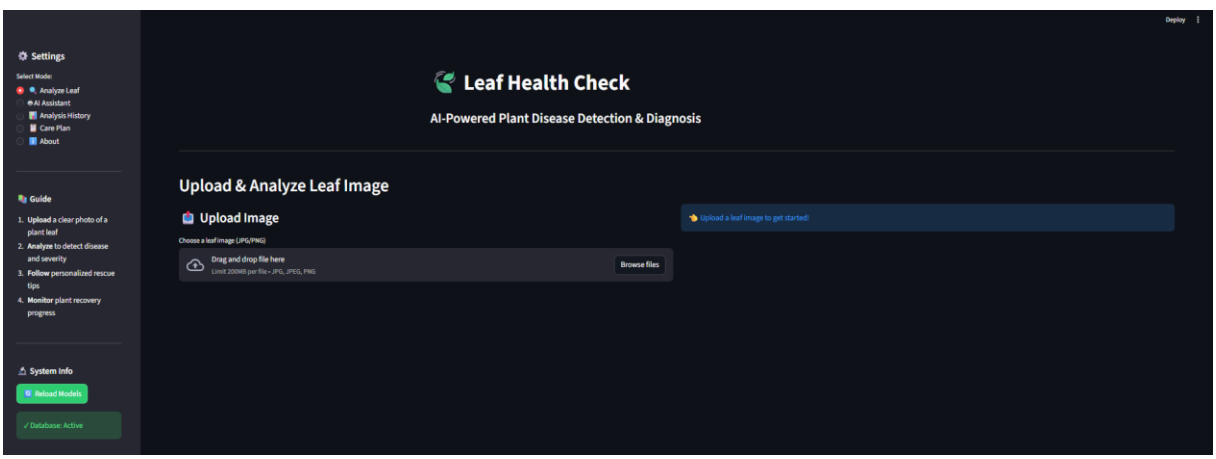
Error Handling and Response Formatting

```
def clean_response(text):
    if text:
        return text.strip()
    return "No response generated"
```

MILESTONE 4: UI Development

This milestone focuses on designing and implementing a **user-friendly and responsive interface** for the **Leaf Health Check** application using the **Streamlit framework**. The UI ensures smooth interaction between the user and the AI system, enabling easy image upload, analysis execution, and result visualization.

Home Page Development



The home page acts as the primary interface for user interaction and provides clear guidance on how to use the application.

Home Page Features:

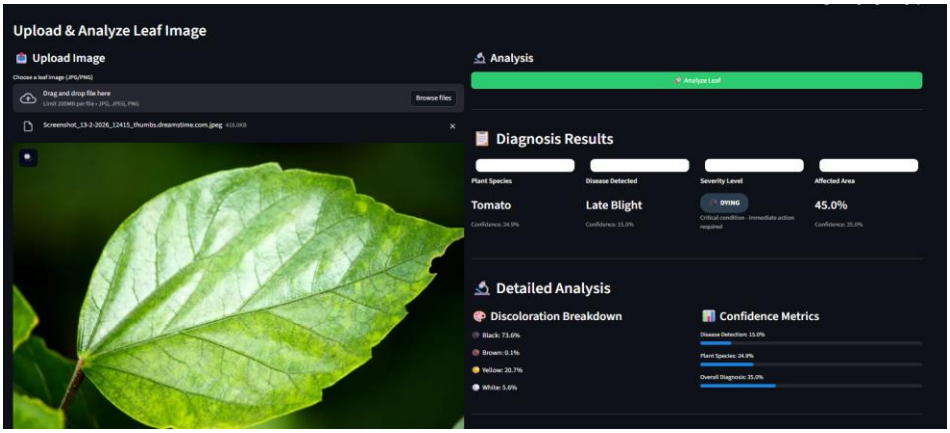
- Application title and description
- Leaf image upload section

- Supported image format information
- Analyze button for triggering diagnosis
- Clean and responsive layout

UI Design Considerations:

- Simple and minimal design
- Clear instructions for users
- Responsive layout across screen sizes
- Intuitive navigation flow

Leaf Analysis Page



This section handles user input and displays AI-generated results after analysis.

Features Implemented:

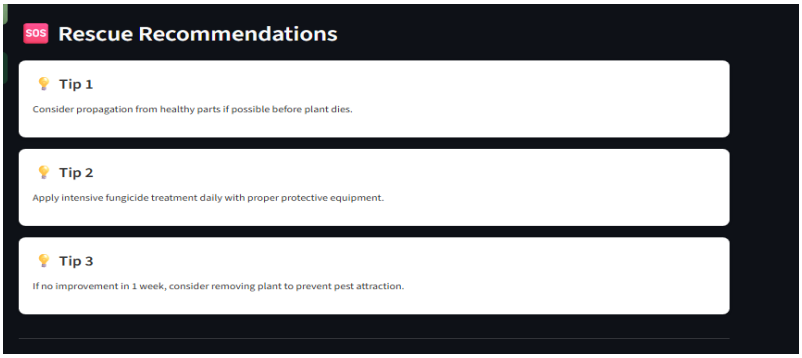
- Image preview after upload
- Loading spinner during analysis
- Gemini AI diagnosis display
- Plant species identification

- Disease detection result
- Severity grading badge

Severity Levels Displayed:

- Healthy
- Mild
- Moderate
- Severe
- Dying

Rescue Recommendation Display

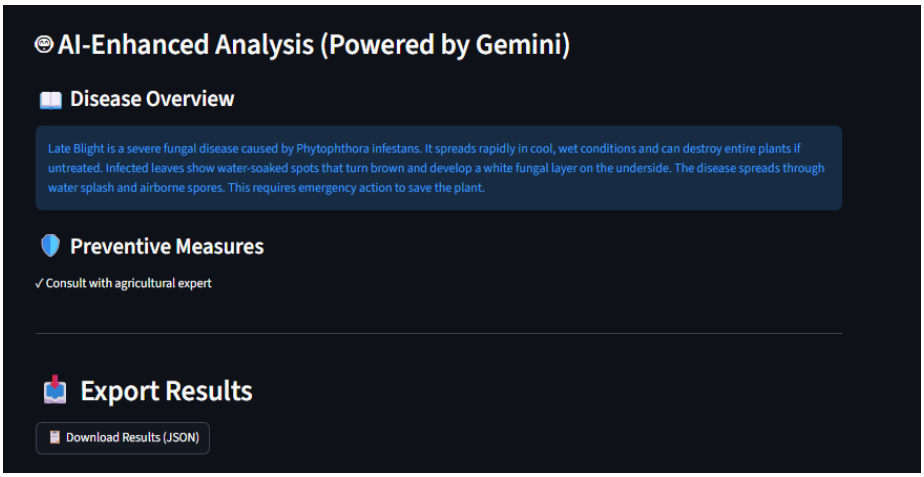


After diagnosis, the system presents **actionable rescue recommendations** to help users treat the affected plant.

Rescue Tips Section Includes:

- Three personalized rescue tips
- Tips based on disease type and severity
- Easy-to-read bullet format
- Practical and actionable suggestions

Result Export Option



To enhance usability, the UI provides an option to download the diagnosis report.

Export Features:

- Download diagnosis as a text/PDF report
- Automatically generated report name
- Includes plant name, disease, severity, and tips

Error Handling and User Feedback

Proper feedback mechanisms are integrated to improve user experience.

Handled Scenarios:

- Invalid image format
- No image uploaded
- AI response failure
- Low-quality image warning

Feedback Methods:

- Warning messages
- Error alerts
- Success notifications

MILESTONE 5: Testing & Optimization

This milestone focuses on **comprehensive testing, quality evaluation, and performance optimization** of the **Leaf Health Check** application to ensure accuracy, reliability, and smooth user experience. Multiple test scenarios were executed to validate image analysis, AI diagnosis, severity grading, and rescue recommendations.

Comprehensive Testing

Test Scenarios

1. Leaf Image Analysis
   - Input Type: Plant leaf images
   - Formats Tested: JPG, PNG, JPEG
   - Conditions: Healthy, diseased, partially damaged leaves
   - Expected Output: Accurate diagnosis with severity grading and rescue tips

Test Cases Executed:

- Healthy green leaf image
- Yellowing leaf image
- Brown spot infected leaf
- Severely damaged leaf image
- Low-quality and blurred images

2. Discoloration Detection
   - Colors Tested: Yellow, brown, black
   - Severity Levels Verified: Healthy, Mild, Moderate, Severe, Dying

- Expected Result: Correct severity classification based on affected area

3. Gemini AI Diagnosis Testing
- API Used: Gemini AI Studio API Key
- Model Tested: Gemini Vision Model
- Expected Output:
    - Plant species identification
    - Disease prediction
    - Confidence-based severity grading
    - Three actionable rescue tips

4. Error Handling Scenarios
- No image uploaded
- Unsupported image format
- Corrupted image file
- API response delay or failure

Expected Behavior:
- User-friendly error messages
- Graceful failure handling
- No application crash

Quality Evaluation

The output quality of the system was evaluated using the following metrics:
- Diagnosis Accuracy
- Severity Classification Correctness
- Relevance of Rescue Tips
- Response Consistency
- User Interface Clarity
- Report Export Quality

Each test output was manually reviewed to ensure **logical correctness** and **practical usefulness**.

Performance Optimization

Improvements Made

- Faster image preprocessing using optimized resizing
- Reduced Gemini API response latency
- Improved input validation logic
- Optimized Streamlit layout rendering
- Added loading indicators for better user feedback
- Reduced unnecessary re-computation using session state

Final Validation

After optimization, the system was re-tested to confirm:

- Stable application performance
- Accurate AI predictions
- Correct severity grading
- Smooth UI interaction
- Successful report downloads

Conclusion:

The Leaf Health Check project demonstrates how artificial intelligence can be used to improve plant health monitoring and disease detection. By combining image processing, deep learning, and an interactive web interface, the system provides a fast and user-friendly solution for diagnosing plant diseases.

The application allows users to upload leaf images and receive instant analysis results, including disease predictions, severity levels, and rescue recommendations. This helps users identify plant problems early and take appropriate actions to protect their plants.

The system has potential applications in agriculture, gardening, and research. With further improvements such as larger datasets and real-time camera integration, the Leaf Health Check system can become a powerful tool for smart agriculture and sustainable farming practices.